

EECS 498 · AASE · FALL 2026

Inside the call: messages, streams, and tool calls

LECTURE 07 · SEP 22, 2026

TODAY

Three parts

1. One POST, one reply
2. The stream
3. Tools, and the loop on the wire

Analyze begins here. Nothing is handed out for three weeks.

LAB01, THREE WEEKS AGO

The card we never opened



Assemble

L02, L03, the repo map



Answer

A line of configuration



Apply

F5, F6, the diff, the commit

Three weeks on the blue cards. Today the amber one.

Three dates, one call

When	What it needs
This week	You write the client. <code>ENDPOINT.md</code> , F1
Thursday evening	An AI capability in <code>task1</code> , two hours
Week 5	The same call, with tools, unattended

One POST

THE REQUEST, FIELD BY FIELD

The request object

POST `{base_url}/chat/completions`

```
{  
  "model": "qwen3.5:9b",  
  "messages": [  
    {"role": "system", "content": "..."},  
    {"role": "user", "content": "..."},  
    {"role": "assistant", "content": "..."},  
    {"role": "user", "content": "..."}  
  ],  
  "stream": true  
}
```

This is the entire interface between your program and the model.

There is **no session field**. There is
nowhere to **put** a memory.

The URL rule

Configured	base_url	Where the request goes
http://localhost:11434/v1		...:11434/v1/chat/completions
http://localhost:11434/v1/		...:11434/v1/chat/completions
https://api.example.edu		...example.edu/chat/completions

Strip one trailing slash, append the path. Never invent a `/v1`.

Three more request fields

- `temperature` — L02's sampling step, as a dial
- `max_tokens` — a cap, and a test's best friend
- `tools` — the second half of today

Omit `temperature` and the server picks. It does not pick zero.

What "compatible" promises

- Ollama, llama.cpp, vLLM, every vendor: same object
- Nobody ratified it. One vendor's API won
- It frays at reasoning, at tools, at `usage`

`ENDPOINT.md` draws the line: text chat completions, nothing more.

One reply, three endings

WHAT COMES BACK, AND WHAT IT OBLIGATES

The reply envelope

```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "HTTP status 404 means the requested resource could not be found on the server.",
      "reasoning": "Thinking Process:\n\n1.  **Analyze the Request:** ..."
    },
    "finish_reason": "stop"
  }],
  "usage": {"prompt_tokens": 31, "completion_tokens": 465, "total_tokens": 496}
}
```

Run it live. One short question, one short answer.

EVERY REPLY ENDS FOR A REASON

Three finish reasons

Value	Your program's obligation
stop	The model ended it. F5 may look for edit blocks
length	Cut off. An explicit error, never an edit
tool_calls	It is asking. Second half

Information for your code, not for the user.

A truncated reply

```
"message": {  
  "role": "assistant",  
  "content": "",  
  "reasoning": "Here's a thinking process that leads to"  
},  
"finish_reason": "length"
```

Empty content. The model did not say nothing.

The number your estimator is graded against

```
"usage": {"prompt_tokens": 31, "completion_tokens": 465, "total_tokens": 496}
```

- `/tokens` predicts. This field counted
- Send once, compare, calibrate. One afternoon

F1 lets the estimate be crude. It does not let it be uncalibrated.

Where reasoning arrives

Server	Where the thinking lands
Ollama <code>/v1</code>	A <code>reasoning</code> field beside <code>content</code>
Others	Inside <code>content</code> , in <code><think></code> tags

Three decisions: show it? keep it in history? is a block in it a proposal?

The stream

STREAM: TRUE, WHICH STAGE 1 ALWAYS IS

A reply as events

```
data: {"choices":[{"delta":{"role":"assistant","content":"","reasoning":"Thinking"}}]}
```

```
data: {"choices":[{"delta":{"content":"","reasoning":" Process"}}]}
```

```
...
```

```
data: {"choices":[{"delta":{"content":" server"}}]}
```

```
data: {"choices":[{"delta":{},"finish_reason":"stop"}]}
```

```
data: [DONE]
```

516 events for one sentence. Most carried no text at all.

Six kinds of event

Event	Carries
Opening delta	The <code>role</code> . Here, a reasoning fragment too
Reasoning-only delta	Thinking. Hundreds of them
Content delta	The text the user sees
Empty delta	The <code>finish_reason</code>
Usage-only event	No <code>choices</code> at all
<code>[DONE]</code>	Not JSON. The normal end

A stream that dies at event 400 has
already **printed most of a reply.**

The failed-stream rule

Your program	Because
Records no assistant turn	The next request would carry a reply it never finished
Treats nothing as an edit	F5 and F6 rest on the proposal being complete
Says so, stays usable	The user has to be able to retry

The one clause of the contract with a safety consequence.

LAB00'S BIN/CHECK HAD A LINE ABOUT THIS

The truncation nobody reports

- Ollama keeps 4k tokens unless told otherwise
- Over that: no error, no `length`, no message
- It drops the **start**. Your system prompt, your first file

Raise it to 32k. Your 24000 budget fits with room for the reply.

The 451 tokens you did not see

That reply billed **465** completion tokens. **14** of them were the sentence.

Your program appends that turn to history and charges it against `context_budget` .

Write one line: `charged: <number>, reasoning: keep | drop`

Then be ready to say what your choice costs on the next request.

The tool envelope

THE SAME FOUR PARTS, ON THE WIRE

The four parts of an action

Part	Whose
Name and description	The model's. What it reads to choose
Argument contract	Yours. Code that checks the ask
Dispatch	Yours. Code that does it
Observation	Yours. The next message in the list

Describing a tool gives the model the ability to ask. Nothing more.

A tool definition

```
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "Read a UTF-8 text file from the repository.",
    "parameters": {
      "type": "object",
      "properties": {"path": {"type": "string"}},
      "required": ["path"]
    }
  }
}
```

The three the Analyze spec names: `read_file`, `write_file`, `run_command`.

MEASURED, SAME THREE MESSAGES

What a tool definition costs

Request	prompt_tokens
Without tools	71
With three tools	471

400 tokens, every request, for the rest of the run.

The reply is a request

```
"message": {  
  "role": "assistant",  
  "content": "",  
  "tool_calls": [{  
    "id": "call_kk9dbt2h",  
    "type": "function",  
    "function": {"name": "read_file", "arguments": "{\"path\":\"taskr/cli.py\"}"}}  
  ]  
},  
"finish_reason": "tool_calls"
```

It has not answered. It has asked.

EACH OF THESE BITES ONCE

Four things in that reply

Field	What it costs you
<code>content: ""</code>	Empty is not done. Branch on <code>finish_reason</code>
<code>arguments</code>	A string of JSON the model wrote
<code>id</code>	How the result gets back. Keep it
<code>tool_calls</code>	A list. It may ask for three at once

TWO APPENDS, THEN SEND

The round trip

```
{"role": "assistant", "content": "",  
  "tool_calls": [ ...exactly as received... ]},  
  
{"role": "tool", "tool_call_id": "call_kk9dbt2h",  
  "content": "\"\"\"Argument parsing and command dispatch for the taskr CLI.\"\"\"n..."}
```

The first line is the one everybody forgets.

Native calls or a text protocol

Native tools

A role, an id, a schema the server saw

Fragmented arguments on some servers

Required in week 5

JSON in prose

Works on any model at all

You write the parser

Fenced JSON, prose around it

Parts two, three and four are the same code either way.

The loop on the wire

L06'S SIX LINES, RUN FOR REAL

The loop from lecture

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text
    for call in reply.tool_calls:
        result = execute(call)
        messages.append(result)
```

Unchanged from Thursday. You now know what every line sends.

The same six lines, as requests

The line	On the wire
<code>model.call(..., tools=)</code>	The POST, plus 400 tokens
<code>if not reply.tool_calls</code>	Branch on <code>finish_reason</code>
<code>execute(call)</code>	Parts two and three. Later: approval
<code>messages.append(result)</code>	Two appends, not one

The run, on your own repository

#	Tool	Argument	tokens
1-3	read_file	cli.py , task.py , store.py	471 → 1214
4-6	write_file	task.py , store.py , cli.py	1638 → 2454
7	run_command	python -c "from cli import main; ..."	3097
8-9	run_command	rm -f ~/.taskr.json && ...	3168 → 3252

Nothing was executed. Every `run_command` was told `exit 0`.

What it got right

- Read `cli.py`, then `store.py`, then `task.py`
- The correct three files. A flag touches all of them
- Wrote them in dependency order, not read order

A 9B read before it wrote, and sequenced its writes.

`rm -f ~/.taskr.json` – and **git** never
knew that file existed.

Context it chose to add

Step	prompt_tokens
1	471
4	1638
9	3252

Sevenfold in nine steps, and it never said it was done.

What actually failed

Nothing	Fix
Stood between <code>rm -f</code> and <code>\$HOME</code>	Approval, in code
Recognised completion	Six named stop conditions
Made a command's result real	A <code>run_command</code> that runs
Capped the growing context	A token budget, not just a step limit

The model read carefully. The harness is what was missing.

TONIGHT

ENDPOINT.md as a checklist

- The URL rule, conditional auth, the request object
- Six event kinds, three finish reasons, reasoning
- The failed-stream rule

Every one is a test you can write against a fake stream. No endpoint needed.

Make the call a **parameter with a default**.

```
def plan(task, call=complete)
```

The week ahead

When	What
Today, after this	The briefing for tonight
Tonight, 7 to 9 PM	Hackathon 1, Leinweber 1355
Tue Oct 6	The build is due. Analyze releases

Nothing about Analyze is handed out before October 6.

TAKE THESE WITH YOU

Three questions

1. Stream dies after forty words. What is in `messages` ?
2. Three tools cost 400 tokens. At how many does 24000 break?
3. It wrote `store.py`'s convention unread. Prose or code?

COMING UP

What the six lines are missing

- Six reasons a run ends, each named in the log
- The prompt in front of an action you did not read

Next: the briefing for tonight.

Questions?